
0G LockControl

Diseño lógico del firmware:
entradas, estados, eventos y payload

Alarma IoT dual — módulo **LSM110A** (STM32WL55 + Sigfox RC2)

Autor: José Francisco Díaz

Área: I+D / Ingeniería — 0G IoT Solutions

Proyecto: 0G-LockControl-LSM110A

Documento: Especificación de máquina de estados y clasificación de eventos

Versión: 0.1 (borrador de diseño)

Fecha: 23 de julio de 2026

Documento de diseño. Complementa a WP-01 «Codec del payload». Cualquier cambio al contrato de bytes descrito aquí debe reflejarse en `payload_codec.{h,c}`, `payload_parser.py` y los vectores de prueba.

Índice

1	Resumen ejecutivo	2
2	Entradas del sistema	2
2.1	Premisa de operación (la regla de oro)	3
3	Casos de uso	3
4	Variables de estado temporales	3
5	Eventos y tipos de mensaje	4
5.1	Eventos internos (disparadores de la FSM)	4
5.2	Tipos y subtipos de mensaje	4
6	Máquina de estados (FSM)	5
7	Lógica de clasificación: tabla de verdad y Karnaugh	5
7.1	¿Qué otros casos podría haber?	7
8	Análisis temporal: ventana de 10 s vs. 1 minuto	7
8.1	Por qué la ventana es de 10 s y no de 60 s	7
9	Medición del “tiempo abierto” y consumo	8
9.1	La respuesta corta	8
9.2	El porqué, con números	8
10	Secuencia de operación	9
11	Propuesta de payload (extensión de WP-01)	10
12	Variables de estado y control para el firmware	10
12.1	Variables de estado (RAM, cambian en runtime)	11
12.2	Banderas de evento (<i>volatile</i> , puestas por ISR/EXTI)	11
12.3	Parámetros de control (config en flash)	11
13	Decisiones y descartes	12
14	Conclusiones y siguientes pasos	12

1 Resumen ejecutivo

Este documento le da estructura y coherencia a la lógica del firmware de **0G LockControl**. Parte de tres entradas físicas (botón de armado, sensor Hall de apertura y acelerómetro) y define de forma cerrada: las **variables de estado**, los **eventos**, la **máquina de estados (FSM)**, la **lógica de clasificación** del evento (resuelta con mapa de Karnaugh) y el **mensaje de salida** con sus tres leyendas: APERTURA, CIERRE y VANDALISMO.

Las decisiones de diseño clave son:

- **Premisa de guardado:** el dispositivo sólo transmite si está **armado**. Si el usuario no presionó el botón al salir, la puerta puede abrirse y cerrarse sin generar ningún mensaje.
- **Ventana de observación $T_{OBS} = 10$ s:** tras la primera apertura se abre una ventana durante la cual se cuentan las aperturas (N) y se mide el desplazamiento máximo del acelerómetro (θ_{max}). Al cierre de la ventana se clasifica y se transmite *un solo* mensaje.
- **Clasificación por 3 variables booleanas** (C, M, P) que produce exactamente una de tres salidas mutuamente excluyentes (Sección 7).
- **Medir «tiempo abierto» SÍ es viable** y se conserva, siempre que se implemente con el **RTC** (marca de tiempo en cada flanco) y *no* con un timer de propósito general manteniendo el MCU despierto (Sección 9). El acercamiento ingenio se **descarta** y se justifica el porqué.
- **La hora no viaja en el payload:** la aporta el backend Sigfox. El firmware sólo mide *duración*.

2 Entradas del sistema

El dispositivo tiene tres fuentes de entrada, todas ya contempladas en el pinout del LSM110A:

Entrada	Componente	Pin / IRQ	Rol en la lógica
Botón de inicio	Push-button	PA? / EXTI	Arma / desarma el sistema. Al salir de casa, la persona lo presiona. Es la llave maestra: sin armado, no hay mensajes.
Apertura	Reed switch + DRV5032 (Hall)	PA1 / EXTI1	Detecta el flanco de apertura/cierre de la puerta. Fuente primaria del evento y del conteo N .
Movimiento	LIS2DW12 (acelerómetro I ² C)	PA0 / EXTI0 (INT1)	Mide el desplazamiento angular θ respecto a la posición de reposo (<code>accel_ref</code>). Discrimina apertura real de forcejeo.

Nota. El botón no aparece todavía en el pinout de la skill del proyecto; hay que asignarle un GPIO con EXTI libre (p.ej. un PBx no usado) y su antirrebote. El acelerómetro puede además usarse como fuente de *wake* por umbral (INT1), lo que permite que el MCU duerma en Stop2 hasta que algo se mueva.

2.1 Premisa de operación (la regla de oro)

“Sólo se transmite si el sistema está armado.”

Esto se modela como una **guarda global**: cualquier flanco del sensor Hall o del acelerómetro se ignora mientras `estado_fsm == DESARMADO`. Es lo que evita que el ir y venir normal de la casa (cuando hay gente adentro) inunde el enlace Sigfox.

3 Casos de uso

La Figura 1 resume los actores y casos de uso. El **usuario** arma y desarma; el **intruso** es un actor secundario que dispara las detecciones; el **backend Sigfox** es el destino de todo mensaje. Todos los casos de detección y el heartbeat culminan en «Enviar mensaje Sigfox» (relación «include»).

0G LockControl — Casos de uso

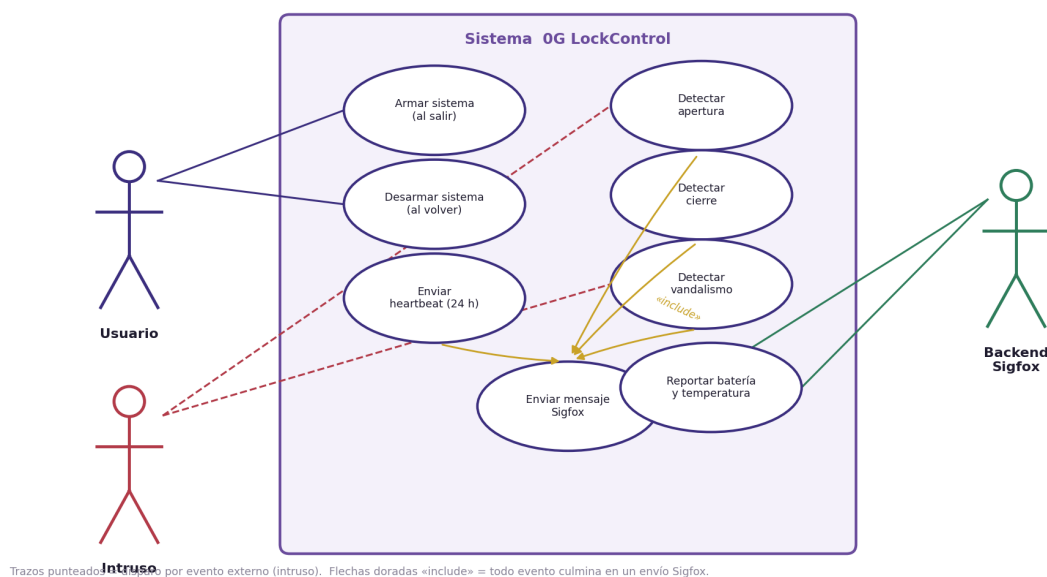


Figura 1: Diagrama de casos de uso de 0G LockControl.

4 Variables de estado temporales

La lógica necesita mantener un pequeño conjunto de variables de estado durante la ventana de observación:

Variable	Símbolo	Significado
Estado de la puerta	P	Abierta (1) / cerrada (0). Se actualiza en cada flanco Hall.
Conteo de aperturas	N	Número de flancos de apertura dentro de la ventana.
Desplazamiento máx.	θ_{max}	Máxima desviación del acelerómetro respecto a accel_ref durante la ventana.
Referencia de reposo	accel_ref	Vector de gravedad capturado al armar (puerta cerrada). Es el “cero” contra el que se mide θ .
Marca de apertura	t_0	Valor del RTC en el flanco de apertura (para calcular el tiempo abierto).
Tiempo abierto	t_{ab}	Duración que la puerta permaneció abierta (sólo si hubo cierre).

De estas variables se derivan las tres **booleanas de decisión** que alimentan la clasificación:

$$C = (N > N_{umbral}), \quad N_{umbral} = 5 \quad \text{("muchos ciclos")} \quad (1)$$

$$M = (\theta_{max} \geq \theta_{umbral}) \quad \text{("movimiento significativo")} \quad (2)$$

$$P = \text{puerta abierta al cierre de la ventana} \quad \text{(estado final)} \quad (3)$$

5 Eventos y tipos de mensaje

5.1 Eventos internos (disparadores de la FSM)

Evento	Origen / efecto
EV_BOTON	EXTI del botón — alterna armado/desarmado.
EV_HALL	EXTI del Hall — flanco de apertura o cierre.
EV_ACCEL	INT1 del acelerómetro — movimiento por umbral.
EV_T_OBS	Fin de la ventana de observación (10 s) — dispara la clasificación.
EV_HEARTBEAT	Timer de 24 h — mensaje de vida.

5.2 Tipos y subtipos de mensaje

En el payload actual, el byte 0 (**tipo**) ya distingue **alarma** (0x01) de **heartbeat** (0x02). Lo que *falta* es codificar la **leyenda** de la alarma. Se propone un byte de **subtipo/evento** (Sección 11):

Mensaje	Código	Cuándo
APERTURA	evento=0x01	La puerta quedó abierta al cierre de la ventana.
CIERRE	evento=0x02	La puerta se abrió y volvió a cerrarse dentro de la ventana. Lleva t_{ab} .
VANDALISMO	evento=0x03	Muchos flancos (C) pero sin apertura real ($\overline{M}$): forcejeo.
heartbeat	tipo=0x02	Cada 24 h; sólo batería/temperatura.

6 Máquina de estados (FSM)

La Figura 2 muestra la máquina completa. En reposo el dispositivo vive en **ARMADO** dentro de **Stop2** ($\sim 3 \mu\text{A}$), y sólo despierta por un flanco Hall, por el acelerómetro o por el timer de heartbeat.

0G LockControl — Máquina de estados (FSM)

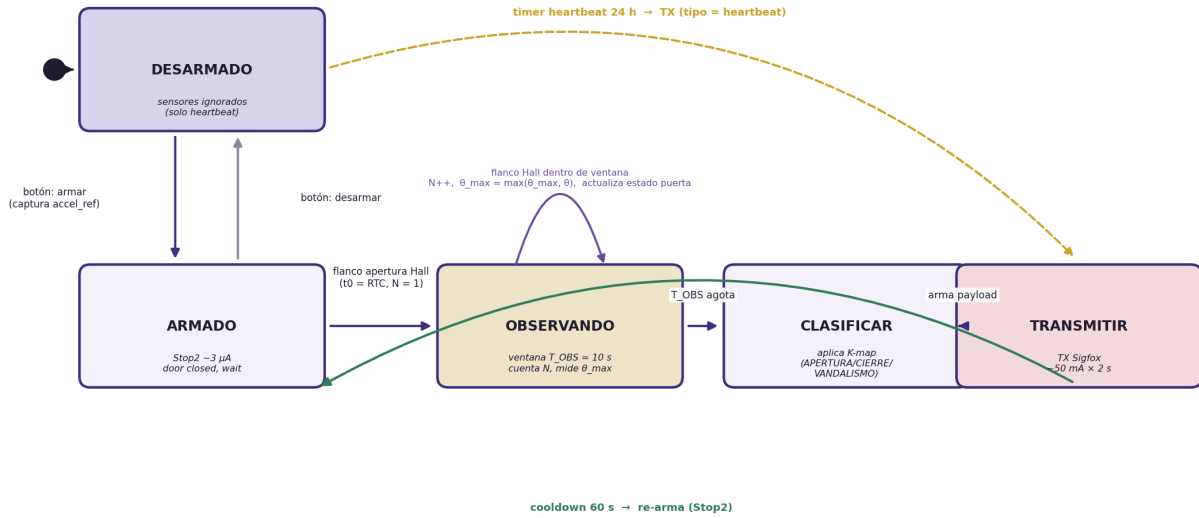


Figura 2: Máquina de estados del firmware. La ventana de observación (T_{OBS}) sólo se arma tras una apertura estando **ARMADO**.

DESARMADO

Sensores ignorados (guarda de la premisa). Sólo corre el heartbeat. Se pasa a **ARMADO** con el botón, capturando `accel_ref`.

ARMADO

Reposo de bajo consumo. Un flanco de apertura del Hall arranca la ventana ($t_0 = \text{RTC}$, $N = 1$) y pasa a **OBSERVANDO**.

OBSERVANDO

Durante $T_{OBS} = 10 \text{ s}$: cada flanco Hall incrementa N y actualiza P ; cada muestra del acelerómetro actualiza θ_{max} . Al agotarse la ventana pasa a **CLASIFICAR**.

CLASIFICAR

Aplica la lógica de la Sección 7 y arma el payload.

TRANSMITIR

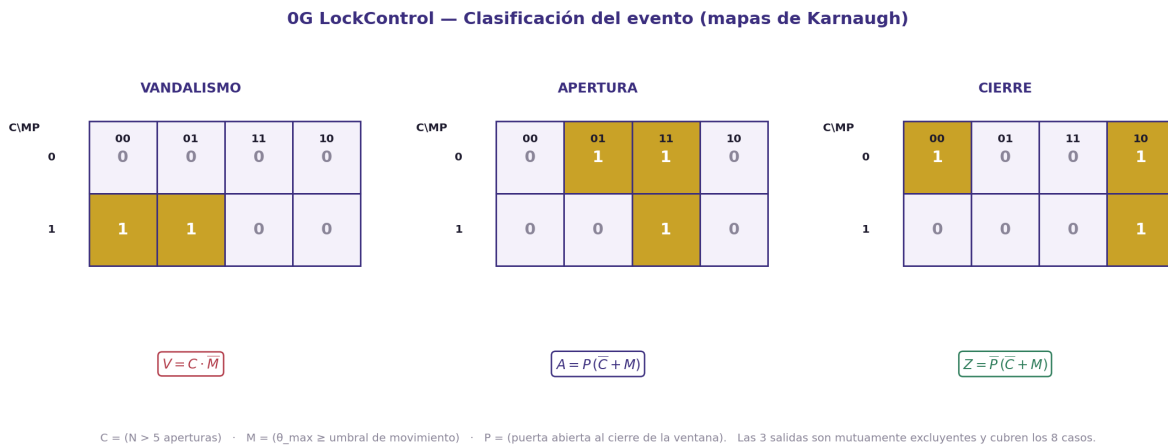
TX Sigfox ($\sim 50 \text{ mA} \times 2 \text{ s}$). Aplica el **cooldown de 60 s** y re-arma (vuelve a **ARMADO**/Stop2).

7 Lógica de clasificación: tabla de verdad y Karnaugh

La decisión de qué leyenda enviar depende de las tres booleanas (C, M, P). Con 3 variables hay $2^3 = 8$ combinaciones. La Tabla 1 las lista todas y la Figura 3 muestra los mapas de Karnaugh.

Tabla 1: Tabla de verdad de la clasificación. $C = (N > 5)$, $M = (\theta_{max} \geq \theta_{umbral})$, P = puerta abierta al final.

C	M	P	Salida	Interpretación
0	0	0	CIERRE	apertura suave, cerró (sin movimiento fuerte)
0	0	1	APERTURA	se abrió y quedó abierta
0	1	0	CIERRE	entrada/salida normal (abrió y cerró)
0	1	1	APERTURA	se abrió de verdad y quedó abierta
1	0	0	VANDALISMO	muchos flancos, sin apertura real, cerrada
1	0	1	VANDALISMO	muchos flancos, sin apertura real
1	1	0	CIERRE	muchos flancos pero abrió de verdad y cerró
1	1	1	APERTURA	muchos flancos y abrió de verdad, quedó abierta

**Figura 3:** Mapas de Karnaugh de las tres salidas. Al simplificar se obtienen expresiones muy compactas que se traducen directo a condicionales en C.

Las ecuaciones minimizadas son:

$$\text{VANDALISMO : } V = C \cdot \bar{M} \quad (4)$$

$$\text{APERTURA : } A = P \cdot (\bar{C} + M) \quad (5)$$

$$\text{CIERRE : } Z = \bar{P} \cdot (\bar{C} + M) \quad (6)$$

El criterio (la “RCO”) es claro: **vandalismo** = muchos flancos del Hall *sin* que el acelerómetro confirme que la puerta realmente se abrió. Es decir, alguien forcejeó la puerta cerrada: el reed switch “castañeteó” varias veces pero la hoja no giró (θ_{max} bajo).

Nota. Con sólo 3 variables la minimización es casi trivial; el valor del Karnaugh aquí es *documental*: deja por escrito y verificable que las tres salidas cubren los 8 casos y son mutuamente excluyentes, sin huecos ni solapes.

En C, la clasificación es literalmente:

```

1 evento_t clasificar(uint16_t N, uint16_t th_max, bool puerta_abierta) {
2     bool C = (N > N_UMBRAL);           /* muchos ciclos */
3     bool M = (th_max >= THETA_UMBRAL); /* movimiento real */

```

```

4      bool P = puerta_abierta;                                /* estado final */
5
6      if (C && !M) return EV_VANDALISMO;                      /* V = C . !M */
7      if (P)      return EV_APERTURA;                        /* A = P . (!C + M) */
8      else       return EV_CIERRE;                           /* Z = !P . (!C + M) */
9  }

```

Listing 1: Clasificación derivada del Karnaugh.

7.1 ¿Qué otros casos podría haber?

Además de las tres leyendas y el heartbeat, quedan como **casos futuros** (fuera del alcance inicial, pero fáciles de encajar en el mismo byte **evento**):

- **Batería baja** — cuando `bateria_pct` cruza un umbral; puede piggy-back en el heartbeat en vez de gastar un mensaje propio.
- **Sabotaje del dispositivo (tamper)** — golpe/arranque de la pared detectado por el acelerómetro con la puerta *cerrada*. Es distinto del vandalismo de puerta.
- **Puerta abierta prolongada** — recordatorio si t_{ab} supera un límite; usar con cuidado por el presupuesto Sigfox.

8 Análisis temporal: ventana de 10 s vs. 1 minuto

Hay que separar **dos temporizadores distintos** que es fácil confundir:

Temporizador	Valor	Propósito
Ventana de observación T_{OBS}	10 s	Agrega la ráfaga de flancos en <i>un</i> evento, confirma que la apertura es real y mide N y θ_{max} antes de decidir.
Cooldown de transmisión	60 s (ya decidido)	Tiempo mínimo entre transmisiones. Es el <i>anti-flood</i> que protege el presupuesto Sigfox (130 msg/día).

8.1 Por qué la ventana es de 10 s y no de 60 s

La ventana cumple cuatro funciones: (1) **descartar glitches** (un flanco aislado no es intrusión), (2) **agregar** una ráfaga en un solo mensaje, (3) **capturar el forcejeo** (5 flancos en 10 s = uno cada 2 s, cadencia realista de alguien sacudiendo la puerta) y (4) **detectar el cierre** dentro de la misma ventana para mandar CIERRE con t_{ab} .

- **A favor de 10 s: latencia baja.** En una alarma de seguridad quieres enterarte rápido; 10 s es un buen compromiso entre “confirmar el evento” y “avisar pronto”. Además, como el *anti-flood ya lo maneja el cooldown de 60 s* (decisión existente del proyecto) y el tope `daily_msg_limit=130`, **la ventana no necesita ser larga** para cuidar el presupuesto.
- **En contra de 60 s:** retrasaría la alerta hasta un minuto y acoplaría “observación” con “limitación de tasa”, que son cosas distintas. Sólo tendría sentido en una puerta muy ruidosa (muchas micro-aperturas legítimas) donde quieras promediar patrones más largos.

Recomendación: $T_{OBS} = 10$ s, **parametrizable** (`t_obs_ms` en la config) por si en campo conviene ajustarlo. El “análogo de 1 minuto” que mencionas equivale a fusionar la ventana con

el cooldown; no se recomienda porque degrada la latencia sin ganar nada (el presupuesto ya está protegido).

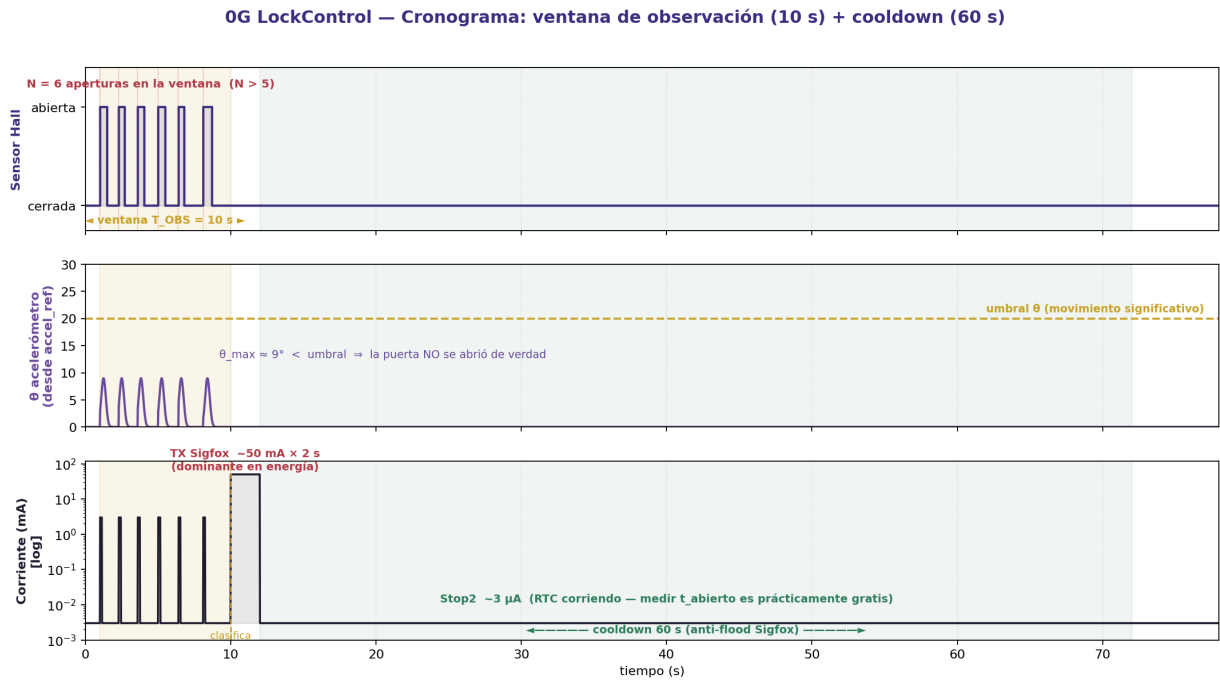


Figura 4: Cronograma del escenario de vandalismo: 6 flancos del Hall en la ventana de 10 s, θ por debajo del umbral (la puerta no se abrió de verdad) $\Rightarrow$ VANDALISMO. Nótese que el pico de energía es la transmisión Sigfox; el reposo en Stop2 es prácticamente plano.

9 Medición del “tiempo abierto” y consumo

Preguntas dos cosas concretas: (a) ¿conviene guardar *cuánto tiempo permaneció abierta* la puerta? y (b) ¿arrancar un timer al abrir para esperar el otro flanco descarga la batería?

9.1 La respuesta corta

Sí conviene medir t_{ab} , y sí es viable — siempre que se use el RTC, no un timer de propósito general con el MCU despierto. La diferencia de consumo es de **tres órdenes de magnitud**.

9.2 El porqué, con números

El proyecto ya opera event-driven: Stop2 ($\sim 3 \mu\text{A}$) $\rightarrow$ wake por INT $\rightarrow$ leer $\rightarrow$ TX ($\sim 50 \text{ mA} \times 2 \text{ s}$) $\rightarrow$ sleep. La clave es *cómo* se mide la duración mientras la puerta está abierta:

- **Enfoque recomendado (RTC + Stop2):** al llegar el flanco de apertura se guarda $t_0 = \text{RTC}$. El MCU **vuelve a dormir en Stop2**. Al llegar el flanco de cierre (EXTI lo despierta) se calcula $t_{ab} = \text{RTC} - t_0$. Durante todo el intervalo el consumo es el de reposo ($\sim 3 \mu\text{A}$), porque el RTC ya está corriendo. Medir cuesta *dos lecturas de registro*: despreciable.
- **Enfoque descartado (timer GP + MCU en Run):** mantener un timer de propósito general contando obliga al MCU a estar despierto (Run/Sleep con reloj rápido), del orden de **mA**, todo el tiempo que la puerta esté abierta.

Tabla 2: Costo de energía de *medir* el tiempo abierto, según el enfoque. Batería CR2450 ≈ 600 mAh (verificar la parte). 1 TX ≈ 0.028 mAh.

Puerta abierta	RTC + Stop2 ($\sim 3 \mu\text{A}$)	Timer GP + Run ($\sim 2 \text{ mA}$)	Factor
1 minuto	$\sim 0.00005 \text{ mAh}$	$\sim 0.033 \text{ mAh}$ ($\approx 1 \text{ TX}$)	$\sim 670\times$
1 hora	$\sim 0.003 \text{ mAh}$	$\sim 2 \text{ mAh}$ ($\approx 72 \text{ TX}$)	$\sim 670\times$
8 horas	$\sim 0.024 \text{ mAh}$	$\sim 16 \text{ mAh}$ ($\approx 576 \text{ TX}$)	$\sim 670\times$

Nota. La corriente en Run depende del reloj (aprox. $\sim 100 \mu\text{A}/\text{MHz}$ para el Cortex-M4); a 16–48 MHz son $\sim 1.6\text{--}5 \text{ mA}$. El número exacto hay que sacarlo del datasheet del STM32WL5x, pero lo robusto es el *orden de magnitud*: mA frente a μA . Si alguien deja la puerta abierta 8 h, el enfoque ingenuo se come $\sim 16 \text{ mAh}$ en *un* evento; el enfoque con RTC, nada.

Conclusión de diseño: se conserva t_{ab} en el payload (byte 11) y se implementa con **captura de RTC en ambos flancos** + Stop2 en medio. El acelerómetro, en modo bajo consumo, ronda unos pocos μA y puede muestrear θ durante la ventana sin impacto relevante. **Se descarta** el timer GP con el MCU despierto.

10 Secuencia de operación

La Figura 5 traza el escenario de vandalismo de punta a punta: desde que el usuario arma al salir, hasta la clasificación y el TX. Las flechas punteadas rojas son eventos asíncronos (EXTI) que despiertan al MCU desde Stop2.

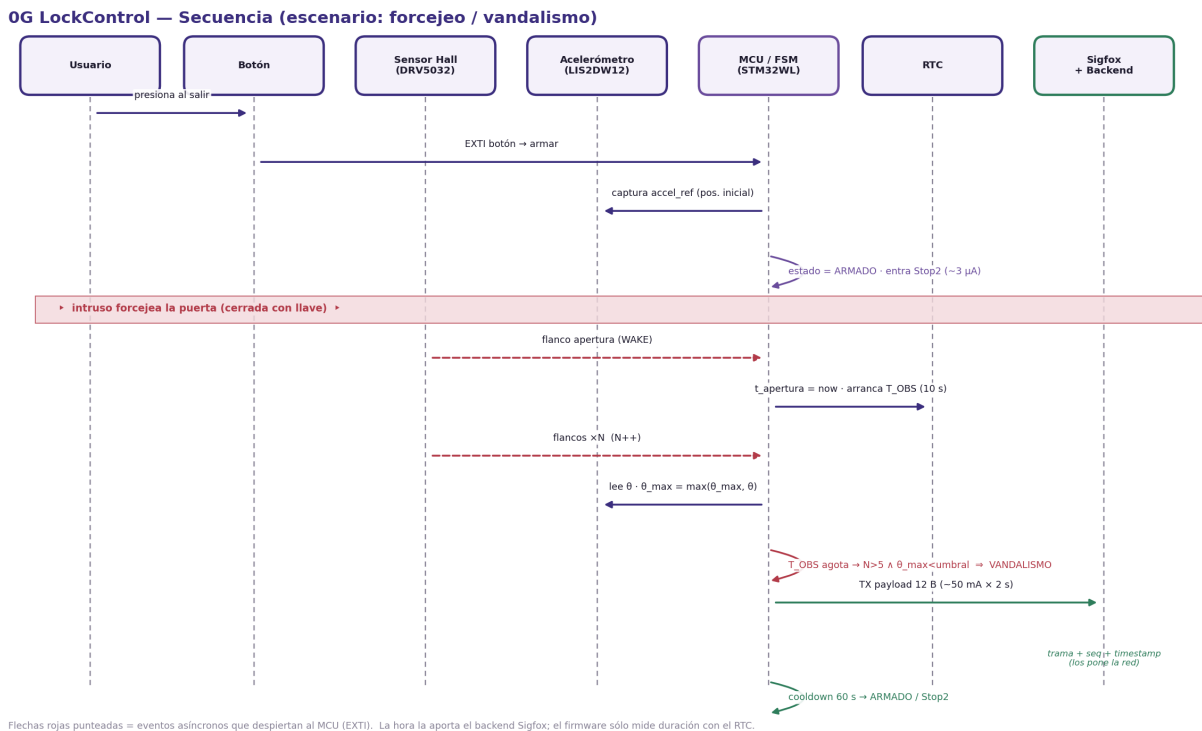


Figura 5: Diagrama de secuencia (escenario: forcejeo / vandalismo).

11 Propuesta de payload (extensión de WP-01)

El payload actual (12 bytes) no tiene dónde poner la **leyenda** ni el **tiempo abierto**. Se propone **reutilizar los dos bytes reservados** (10–11) y **refinar** la semántica de dos campos, sin cambiar la longitud:

Byte	Campo	Estado	Definición propuesta
0	tipo	igual	0x01 alarma / 0x02 heartbeat
1	fuelle	igual	bitfield accel/magnético (qué sensores participaron)
2–3	magnitud_mg	refinado	θ_{max} : máx. desviación vs <code>accel_ref</code> (mg)
4	eje	igual	eje dominante del desplazamiento
5	magnetico	igual	P : 0 cerrado / 1 abierto (estado final)
6	bateria_pct	igual	batería 0..100
7	temp	igual	<code>temp_c</code> + 40
8–9	conteo	redefinido	N : aperturas en la ventana (BE)
10	evento	NUEVO	0x00 ninguno / 0x01 apertura / 0x02 cierre / 0x03 vandalismo
11	t_abierto_s	NUEVO	duración abierta en s (0–255, saturado)

- **Por qué liberar conteo para N :** Sigfox ya entrega un número de secuencia *a nivel de red* (lo ve el backend), así que un contador monotónico en la app es redundante. Ese byte rinde más llevando el conteo de aperturas que pediste para el payload.
- **Compatibilidad:** en el heartbeat los bytes 10–11 quedan en 0x00, igual que hoy, así que los vectores V4/V5 siguen válidos. Los vectores de alarma (V1–V3) habría que recalcularlos para incluir `evento`.

Nota. Este cambio toca el contrato de WP-01. Hay que actualizar `payload_codec.h/.c`, `payload_parser.py` y `test_vectors`, y volver a correr la validación C ↔ Python (5/5).

12 Variables de estado y control para el firmware

12.1 Variables de estado (RAM, cambian en runtime)

Variable	Tipo	Descripción
estado_fsm	fsm_state_t	Estado actual de la FSM.
armado	bool	Sistema armado (guarda global).
puerta_abierta	bool	Estado actual del Hall (P).
n_aperturas	uint16_t	N : flancos de apertura en la ventana.
theta_max	uint16_t	θ_{max} vs accel_ref (mg).
accel_ref[3]	int16_t	Vector de gravedad de reposo (al armar).
t0_rtc	uint32_t	RTC en el flanco de apertura.
t_abierto_s	uint16_t	Duración abierta calculada (s).
t_ventana0	uint32_t	Marca de inicio de la ventana T_{OBS} .
ultimo_tx	uint32_t	Timestamp del último TX (cooldown).
msgs_hoy	uint8_t	Mensajes del día (presupuesto Sigfox).

12.2 Banderas de evento (volatile, puestas por ISR/EXTI)

Bandera	Fuente
flag_boton	EXTI del botón.
flag_hall	EXTI del Hall (+ dirección del flanco).
flag_accel	INT1 del LIS2DW12.
flag_ventana	Fin de T_{OBS} (alarma RTC / LPTIM).
flag_heartbeat	Timer de 24 h.

12.3 Parámetros de control (config en flash)

Extienden la `device_config_t` que ya existe en el proyecto:

```

1  typedef enum {
2      ST_DESARMADO = 0,
3      ST_ARMADO,
4      ST_OBSERVANDO,
5      ST_CLASIFICAR,
6      ST_TRANSMITIR
7  } fsm_state_t;
8
9  typedef enum {
10     EV_NINGUNO      = 0x00,
11     EV_APERTURA    = 0x01,
12     EV_CIERRE       = 0x02,
13     EV_VANDALISMO  = 0x03
14 } evento_t;
15
16 typedef struct {
17     /* --- ya existentes --- */
18     uint16_t accel_threshold_mg; /* 200 -> tambien sirve como THETA_UMBRAL */
19     uint16_t cooldown_seconds; /* 60 -> anti-flood entre TX */
20     int8_t tx_power_dbm; /* 14 */
21     uint8_t heartbeat_hours; /* 24 */
22     uint8_t daily_msg_limit; /* 130 */
23     /* --- NUEVOS para esta logica --- */

```

```

24     uint16_t t_obs_ms;           /* 10000 -> ventana de observacion */
25     uint8_t n_umbral;           /* 5      -> umbral de ciclos (vandalismo) */
26     uint8_t debounce_ms;       /* 30-50 -> antirrebote del Hall/reed */
27 } device_config_t;

```

Listing 2: Config extendida y enums de la FSM.

Nota. El **antirrebote del Hall** (`debounce_ms`) no es opcional: un reed switch rebota, y sin filtrar, *una* apertura real podría contarse como varios flancos y disparar un “vandalismo” falso. Es la salvaguarda que hace confiable la variable N .

13 Decisiones y descartes

Tema	Decisión	Razón
Guarda de armado	Sí	Evita inundar Sigfox con el tránsito normal.
Ventana T_{OBS}	10 s	Latencia baja + agrega ráfaga; el presupuesto ya lo cuida el cooldown.
Ventana de 1 min	Descartada	Retrasa la alerta y acopla observación con rate-limit.
Cooldown	60 s	Anti-flood (ya decidido en el proyecto).
Umbral de ciclos N	5	Cadencia realista de forcejeo.
Umbral θ	Reusar 200 mg	Discrimina apertura real; calibrar en campo.
Medir t_{ab}	Sí, vía RTC	Costo despreciable ($\sim 3 \mu A$).
Timer GP + MCU en Run	Descartado	Consumo en mA; descarga la batería (Tabla 2).
Hora en el payload	Descartada	La aporta el backend Sigfox.
Contador monotónico app	Descartado	Sigfox ya da secuencia de red; se libera conteo para N .

14 Conclusiones y siguientes pasos

El sistema queda cerrado y coherente: tres entradas, una guarda de armado, una ventana de 10 s que agrega y caracteriza el evento, y una clasificación de tres salidas mutuamente excluyentes verificada con Karnaugh. La medición de tiempo abierto se conserva porque, hecha con el RTC, no cuesta batería.

Pendientes concretos:

1. Asignar el **GPIO del botón** (EXTI libre) en CubeMX y su antirrebote.
2. **Calibrar** θ_{umbral} y `debounce_ms` con la puerta real.
3. Actualizar el **payload** (bytes 10–11 + semántica de 2–3 y 8–9) en **codec** y **parser**, y recalcular vectores V1–V3.
4. Documentar esta lógica en el **Notion Hub** del proyecto y hacer commit al repo 0G-LockControl-LSM110A (parando antes de `git push`, como es la costumbre).